TTF Natural Gas Options — User Manual

This manual covers:

- **Part 1.** Introduction — TTF options, Black-76 vs Bachelier, Greeks
- **Part 2.** `black76_ttf.py` — every function with worked examples
- **Part 3.** `ttf_market_data.py` — market data and vol surface
- **Part 4.** `ttf_hh_spread.py` — TTF/HH spread option (Margrabe 1978)
- **Part 5.** `dashboard_jupyter.ipynb` — section-by-section user guide
- **Part 6.** Financial glossary — every term used in the project

> ***Conventions used throughout the examples*** *- TTF forward:* `F = 30 EUR/MWh` *- Strikes:* `25 / 28 / 30 / 32 / 35 EUR/MWh` *- Black-76 volatility:* `sigma = 0.50` *(50% lognormal) - Bachelier volatility:* `sigma_n = 15 EUR/MWh` *(≈* `F · sigma`*) - Maturity:* `T = 0.25` *year (3 months, ACT/365) - Risk-free rate:* `r = 0.02` *(2%) - Spread (Part 4):* `F_HH = 3 USD/MMBtu`, `fx = 1.08`, `σ_TTF = 60%`, `σ_HH = 50%`, `ρ = 0.35`, `r_usd = 0.045`
>
> *Numerical values shown are rounded to 4 decimal places.*

---

1. Introduction

1.1 What is a TTF option?

The **TTF** (*Title Transfer Facility*) is the Dutch virtual natural gas hub and the European benchmark. A **TTF option** grants the right (without the obligation) to buy (`call`) or sell (`put`) a TTF futures contract at a given exercise price (`strike` , in EUR/MWh) on a given expiration date.

- **Underlying**: TTF futures (one contract = physical delivery over a given month).
- **Style**: European — exercise only at expiry.
- **Quote**: EUR/MWh (1 MWh = 3.4121 MMBtu).
- **Day-count**: ACT/365 for the time `T` to expiry.
- **Calendar**: ICE Endex Dutch TTF — option expiry falls ~5 calendar days before the 1st of the delivery month (rolled back to a business day, excluding NL+UK holidays and the futures LTD).

1.2 Black-76 vs Bachelier

Two pricing models are implemented; the choice depends on the price regime.

Criterion	Black-76 (lognormal)	Bachelier (normal)
Assumption	`dF / F = sigma · dW`	`dF = sigma_n · dW`
Vol expressed in	decimal (`0.50` = 50%)	absolute EUR/MWh (`15` = 15 EUR/MWh)
Negative price allowed	**no** (F must be > 0)	**yes**
Suitable regime	normal conditions (F ≫ 0)	crisis / spreads (F low or < 0)
PCP	`C − P = e^(-rT)·(F − K)`	identical

Rule of thumb: if `F < 2 EUR/MWh` or `F` may go negative (spreads, under-balanced gas), switch to Bachelier. Otherwise, Black-76 is the standard.

At ATM (`K = F`), both models yield very similar prices when `sigma_n ≈ F · sigma`.

1.3 The Greeks

Sensitivities of the option price with respect to the input parameters.

Greek	Definition	Unit	Convention used here
Delta Δ	`∂V / ∂F`	dimensionless	discounted by `e^(-rT)`
Gamma Γ	`∂²V / ∂F²`	per EUR/MWh	identical for call/put
Vega ν	`∂V / ∂σ`	EUR/MWh per unit of vol	per 1.00 (100%) of vol
Theta Θ	`∂V / ∂t`	EUR/MWh per day	per calendar day (negative if long)
Rho ρ	`∂V / ∂r`	EUR/MWh per 1 pp	per percentage point
Vanna	`∂Δ / ∂σ`	mixed	cross derivative
Volga	`∂²V / ∂σ²`	EUR/MWh	vol convexity

Practical conventions:

- `Δ_call − Δ_put = e^(-rT)` (forward delta parity).
- `Γ`, `ν` are identical for call and put under both Black-76 and Bachelier.
- `Θ` here is divided by 365 → **per day**, not per year.
- `ρ` here is divided by 100 → per **percentage point** of rate change.

2. `black76_ttf.py`

A single module, with no external dependency beyond `scipy.stats.norm` and `scipy.optimize.brentq`. It covers:

- The ICE Endex Dutch TTF expiry calendar
- Time-to-expiry helpers (`T` in years, ACT/365)
- A contract code parser (`TTFM26` , `Jun26` , …)
- **Black-76** and **Bachelier** pricing (call / put / dispatcher)
- The full set of Greeks (Δ, Γ, ν, Θ, ρ, vanna, volga) for both models
- **Implied volatility** solvers (Brent)
- **Delta** → **strike** inversion

2.1 ICE Endex expiry calendar

`ttf_expiry_date(contract_month: int, contract_year: int) -> date`

Official TTF option expiry date for a given delivery month.

```
from datetime import date
from black76_ttf import ttf_expiry_date

ttf_expiry_date(6, 2026)   # TTFM26: June 2026 delivery
# -> date(2026, 5, 27)

ttf_expiry_date(1, 2026)   # TTFF26: Dec-25 holiday shifts to Dec 24
# -> date(2025, 12, 24)

ttf_expiry_date(3, 2026)   # TTFH26: March 2026 delivery
# -> date(2026, 2, 24)
```

Algorithm:

1. Candidate = 1st of delivery month − 5 calendar days
2. If not a business day → roll back to the previous business day (NL + UK)
3. If equal to the futures LTD → roll back one additional business day

`ttf_time_to_expiry(contract_month, contract_year, reference=None) -> float`

Time `T` (years, ACT/365, including the reference day). Returns 0 if expiry lies in the past.

```
from datetime import date
from black76_ttf import ttf_time_to_expiry

ttf_time_to_expiry(6, 2026, reference=date(2026, 4, 23))
# -> 0.0959  (35 days / 365)
```

`ttf_next_expiries(n=6, reference=None) -> list[tuple[str, date]]`

The next `n` TTF expiries (ICE codes + dates), sorted ascending.

```
from datetime import date
from black76_ttf import ttf_next_expiries

ttf_next_expiries(3, reference=date(2026, 4, 23))
# -> [('TTFK26', date(2026, 4, 24)),
#     ('TTFM26', date(2026, 5, 27)),
#     ('TTFN26', date(2026, 6, 26))]
```

`ttf_is_business_day(d: date) -> bool`

ICE Endex business day (Mon–Fri, excluding `_ttf_holidays(year)` = NL+UK: 1 January, Good Friday, Easter Monday, 1 May, 25 and 26 December).

```
from datetime import date
from black76_ttf import ttf_is_business_day

ttf_is_business_day(date(2025, 12, 25))   # Christmas
# -> False
ttf_is_business_day(date(2026, 4, 6))     # Easter Monday
# -> False
```

2.2 "Simple" expiry helpers (5 business days before the futures LTD)

These coexist with the ICE calendar for backward compatibility. The functions `b76_price_ttf` / `bach_price_ttf` use them via `t_from_contract`.

`futures_expiry_from_delivery(delivery_year, delivery_month) -> date`

Last business day of the month preceding delivery.

```
from black76_ttf import futures_expiry_from_delivery

futures_expiry_from_delivery(2026, 6)
# -> date(2026, 5, 29)   (Friday)
```

`options_expiry_from_delivery(delivery_year, delivery_month) -> date`

5 business days before the futures LTD.

```
from black76_ttf import options_expiry_from_delivery

options_expiry_from_delivery(2026, 6)
# -> date(2026, 5, 22)
```

`t_from_delivery(delivery_year, delivery_month, reference=None) -> float`

Time `T` (ACT/365) up to `options_expiry_from_delivery`.

```
from datetime import date
from black76_ttf import t_from_delivery

t_from_delivery(2026, 6, reference=date(2026, 4, 1))
# -> 0.1425  (52 days / 365)
```

`t_futures_from_delivery(delivery_year, delivery_month, reference=None) -> float`

Same as above but up to the futures LTD.

```
from datetime import date
from black76_ttf import t_futures_from_delivery

t_futures_from_delivery(2026, 6, reference=date(2026, 4, 1))
# -> 0.1616  (59 days / 365)
```

2.3 Contract code parser

`t_from_contract(contract: str, reference=None) -> float`

Accepts the ICE code (`TTFH26`) or the monthly abbreviation (`Mar26`, `Mar2026`) and returns `T` via `t_from_delivery`.

```
from datetime import date
from black76_ttf import t_from_contract

t_from_contract("TTFM26", reference=date(2026, 4, 1))
# -> 0.1425
t_from_contract("Jun26",  reference=date(2026, 4, 1))
# -> 0.1425
t_from_contract("Mar2026", reference=date(2026, 1, 15))
# -> 0.1233
```

Raises `ValueError` if the format is not recognized.

2.4 Black-76 pricing

Supported ICE month codes: `F G H J K M N Q U V X Z`.

`b76_call(F, K, T, r, sigma) -> float`

```
from black76_ttf import b76_call

b76_call(F=30, K=30, T=0.25, r=0.02, sigma=0.50)
# -> 2.9670   EUR/MWh   (ATM call, 3 months)

b76_call(F=30, K=25, T=0.25, r=0.02, sigma=0.50)
# -> 6.4124   EUR/MWh   (ITM call)

b76_call(F=30, K=35, T=0.25, r=0.02, sigma=0.50)
# -> 1.2197   EUR/MWh   (OTM call)
```

`b76_put(F, K, T, r, sigma) -> float`

```
from black76_ttf import b76_put

b76_put(F=30, K=30, T=0.25, r=0.02, sigma=0.50)
# -> 2.9670   EUR/MWh   (ATM put, 3 months)

b76_put(F=30, K=35, T=0.25, r=0.02, sigma=0.50)
# -> 6.1448   EUR/MWh   (ITM put)
```

> ***PCP check**: for `K = 35`, `C − P = 1.2197 − 6.1448 = −4.9251` while `e^(-0.02·0.25)·(30 − 35) = −4.9750`. Exact difference → PCP holds.*

`b76_price(F, K, T, r, sigma, option_type='call') -> float`

Generic dispatcher:

```
from black76_ttf import b76_price

b76_price(30, 30, 0.25, 0.02, 0.50, "call")   # -> 2.9670
b76_price(30, 30, 0.25, 0.02, 0.50, "put")    # -> 2.9670
```

Raises `ValueError` if `option_type` ∉ `{'call', 'put'}`.

`b76_price_ttf(F, K, contract, r, sigma, option_type='call', reference=None) -> float`

Variant where `T` is derived from a contract code.

```
from datetime import date
from black76_ttf import b76_price_ttf

b76_price_ttf(F=30, K=30, contract="TTFM26",
              r=0.02, sigma=0.50,
              option_type="call",
              reference=date(2026, 4, 1))
# -> 2.2530   EUR/MWh   (T = 52 / 365 = 0.1425)
```

2.5 Bachelier pricing

For forwards close to zero or negative, or for spreads.

`bach_call(F, K, T, r, sigma_n) -> float`

```
from black76_ttf import bach_call

bach_call(F=30, K=30, T=0.25, r=0.02, sigma_n=15)
# -> 2.9770   EUR/MWh

bach_call(F=-3, K=0, T=60/365, r=0.02, sigma_n=6)
# -> 0.5860   EUR/MWh   (negative forward, crisis scenario)
```

`bach_put(F, K, T, r, sigma_n) -> float`

```
from black76_ttf import bach_put

bach_put(F=30, K=30, T=0.25, r=0.02, sigma_n=15)
# -> 2.9770
```

`bach_price(F, K, T, r, sigma_n, option_type='call') -> float`

Dispatcher.

```
from black76_ttf import bach_price

bach_price(30, 35, 0.25, 0.02, 15, "put")
# -> 7.6770
```

`bach_price_ttf(F, K, contract, r, sigma_n, option_type='call', reference=None) -> float`

Same as `b76_price_ttf` but using Bachelier.

```
from datetime import date
from black76_ttf import bach_price_ttf

bach_price_ttf(F=30, K=30, contract="Jun26",
               r=0.02, sigma_n=15,
               option_type="call",
               reference=date(2026, 4, 1))
# -> 2.2519
```

2.6 Black-76 Greeks

All the functions below take `(F, K, T, r, sigma, option_type='call')` except `b76_gamma`, `b76_vega`, `b76_vanna`, `b76_volga` which are identical for call and put (no `option_type` argument).

```
from black76_ttf import (
    b76_delta, b76_gamma, b76_vega, b76_theta, b76_rho,
    b76_vanna, b76_volga, b76_greeks,
)

F, K, T, r, sigma = 30, 30, 0.25, 0.02, 0.50

b76_delta(F, K, T, r, sigma, "call")   # -> +0.5470
b76_delta(F, K, T, r, sigma, "put")    # -> -0.4480
b76_gamma(F, K, T, r, sigma)           # -> +0.0525
b76_vega (F, K, T, r, sigma)           # -> +5.9069   (per 1.00 of vol)
b76_theta(F, K, T, r, sigma, "call")   # -> -0.0245   (per day)
b76_rho  (F, K, T, r, sigma, "call")   # -> -0.0074   (per 1 pp)
b76_vanna(F, K, T, r, sigma)           # -> +0.0984
b76_volga(F, K, T, r, sigma)           # -> -0.1846
```

`b76_greeks(F, K, T, r, sigma, option_type='call') -> B76Greeks`

All the Greeks in a single call (`B76Greeks` dataclass with fields `delta`, `gamma`, `vega`, `theta`, `rho`, `vanna`, `volga`).

```
g = b76_greeks(F=30, K=30, T=0.25, r=0.02, sigma=0.50, option_type="call")
g.delta   # +0.5470
g.gamma   # +0.0525
g.vega    # +5.9069
g.theta   # -0.0245
g.rho     # -0.0074
g.vanna   # +0.0984
g.volga   # -0.1846
```

> ***Vega convention**: per unit of decimal vol (1.00 = 100%). For the "per 1 vol point" sensitivity (1% = 0.01), divide vega by 100.*

> ***Rho convention**: already divided by 100, so directly in EUR/MWh per percentage point of rate change.*

2.7 Bachelier Greeks

Same signatures, with `sigma_n` instead of `sigma`.

```
from black76_ttf import (
    bach_delta, bach_gamma, bach_vega, bach_theta, bach_rho,
    bach_vanna, bach_volga, bach_greeks,
)

F, K, T, r, sigma_n = 30, 30, 0.25, 0.02, 15

bach_delta(F, K, T, r, sigma_n, "call")   # -> +0.4975
bach_delta(F, K, T, r, sigma_n, "put")    # -> -0.4975
bach_gamma(F, K, T, r, sigma_n)           # -> +0.0530
bach_vega (F, K, T, r, sigma_n)           # -> +0.1985
bach_theta(F, K, T, r, sigma_n, "call")   # -> -0.0246
bach_rho  (F, K, T, r, sigma_n, "call")   # -> -0.0074
bach_vanna(F, K, T, r, sigma_n)           # 0.0
bach_volga(F, K, T, r, sigma_n)           # -0.0132
```

> *At ATM under Bachelier, `vanna = 0` exactly (by symmetry in `d = (F-K)/(σ√T)`).*

`bach_greeks(...) -> BachGreeks`

```
g = bach_greeks(30, 30, 0.25, 0.02, 15, "call")
g.delta, g.gamma, g.vega, g.theta, g.rho, g.vanna, g.volga
# (0.4975, 0.0530, 0.1985, -0.0246, -0.0074, 0.0, -0.0132)
```

2.8 Implied volatility solvers

Brent method, `xtol = 1e-8`, max 300 iterations.

`b76_implied_vol(market_price, F, K, T, r, option_type='call', sigma_lo=1e-6, sigma_hi=20.0) -> float`

Round-trip price → IV → sigma:

```
from black76_ttf import b76_call, b76_implied_vol

p = b76_call(30, 30, 0.25, 0.02, 0.50)   # 2.9670
b76_implied_vol(p, F=30, K=30, T=0.25, r=0.02, option_type="call")
# -> 0.5000
```

Raises `ValueError` if `market_price` falls outside the `[intrinsic, sigma_hi]` corridor.

`bach_implied_vol(market_price, F, K, T, r, option_type='call', sigma_lo=1e-6, sigma_hi=500.0) -> float`

Inverse for the normal vol in EUR/MWh.

```
from black76_ttf import bach_call, bach_implied_vol

p = bach_call(30, 30, 0.25, 0.02, 15)   # 2.9770
bach_implied_vol(p, F=30, K=30, T=0.25, r=0.02, option_type="call")
# -> 15.0000
```

2.9 Delta → strike inversion

Useful for converting a 25Δ / 50Δ / 75Δ quote into a strike.

`b76_delta_to_strike(delta_target, F, T, r, sigma, option_type='call', K_lo=None, K_hi=None) -> float`

```
from black76_ttf import b76_delta_to_strike

# Strike of the 25-delta call (typically OTM)
b76_delta_to_strike(delta_target=0.25,
                    F=30, T=0.25, r=0.02, sigma=0.50,
                    option_type="call")
# -> 35.0826   EUR/MWh

# Strike of the 25-delta put (the target delta is negative for a put)
b76_delta_to_strike(delta_target=-0.25,
                    F=30, T=0.25, r=0.02, sigma=0.50,
                    option_type="put")
# -> 25.5749   EUR/MWh
```

Defaults for `K_lo` / `K_hi`: `[F · 0.01, F · 10]`. Raises `ValueError` if the target is not reachable within the bracket.

`bach_delta_to_strike(delta_target, F, T, r, sigma_n, option_type='call', K_lo=None, K_hi=None) -> float`

Bachelier variant — useful for negative or very low forwards.

```
from black76_ttf import bach_delta_to_strike

bach_delta_to_strike(delta_target=0.25,
                     F=30, T=0.25, r=0.02, sigma_n=15,
                     option_type="call")
# -> 35.0570   EUR/MWh
```

Defaults for `K_lo` / `K_hi` : `[F − 10·σ_n·√T, F + 10·σ_n·√T]` .

---

3. `ttf_market_data.py`

End-to-end market data layer for the TTF natural gas curve and its volatility surface. The module is **fully self-contained** (no `yfinance` dependency) and gracefully falls back to a synthetic curve when the public quote feed is unreachable.

What is covered:

- TTF expiry calendar (ICE/EEX conventions)
- Forward curve fetching (Yahoo Finance via `requests`) with synthetic fallback
- Volatility surface construction (strikes × maturities, parametric smile)
- SABR market calibration (Hagan et al. 2002)
- CSV / JSON export for downstream pricing modules

> ***External dependencies*** *:* `numpy` *,* `pandas` *,* `scipy` *,* `requests` *. Make sure* `requirements.txt` *lists* `pandas>=2.0` *and* `requests>=2.31` *in addition to* `numpy` *and* `scipy` *.*

3.1 `TTFContract` dataclass

A single monthly contract description.

Field	Type	Meaning
`delivery_month`	`int`	1–12
`delivery_year`	`int`	e.g. `2026`
`expiry_date`	`date`	Options expiry (5 BD before futures LTD)
`futures_expiry_date`	`date`	Futures last trading day
`contract_code`	`str`	ICE-style code, e.g. `"TTFH26"`
`T`	`float`	Time to options expiry in years (ACT/365, today included)

3.2 `TTFExpiryCalendar`

Manages the ICE/EEX TTF calendar starting from an arbitrary reference date.

```
from datetime import date
from ttf_market_data import TTFExpiryCalendar

cal = TTFExpiryCalendar(reference_date=date(2026, 4, 20))
```

`futures_expiry_date(delivery_year, delivery_month) -> date`

Last business day of the month preceding delivery.

```
cal.futures_expiry_date(2026, 6)   # TTFM26
# -> date(2026, 5, 29)             (Friday)
```

`expiry_date(delivery_year, delivery_month) -> date`

Options expiry: 5 business days before the futures LTD.

```
cal.expiry_date(2026, 6)
# -> date(2026, 5, 22)
```

`contract_code(delivery_year, delivery_month) -> str`

Returns the ICE-style code (`TTF` + month code letter + last two digits of the year).

```
cal.contract_code(2026, 3)         # -> "TTFH26"
cal.contract_code(2027, 12)        # -> "TTFZ27"
```

`time_to_expiry(expiry) -> float`

Time `T` in years (ACT/365, reference date included). Returns `0` if the expiry is already in the past.

```
cal.time_to_expiry(date(2026, 5, 22))
# -> 0.0904   (33 days / 365)
```

`active_contracts(n=12) -> list[TTFContract]`

The next `n` monthly TTF contracts (sorted ascending), skipping any whose expiry has already passed relative to the reference date.

```
contracts = cal.active_contracts(n=3)
for c in contracts:
    print(c.contract_code, c.expiry_date, round(c.T, 4))
# TTFK26  2026-04-24  0.0137
# TTFM26  2026-05-22  0.0904
# TTFN26  2026-06-23  0.1781
```

`expiry_for_tenor(tenor_years) -> date`

Returns the expiry date of the contract closest to a given tenor expressed in years. Useful when interpolating a vol smile by tenor rather than by contract.

```
cal.expiry_for_tenor(0.50)
# -> date(2026, 10, 23)   (≈ 6 months from 2026-04-20)
```

3.3 `ForwardPoint` dataclass

A single (contract, price) observation on the curve.

Field	Type	Meaning
`contract_code`	`str`	e.g. `"TTFM26"`
`delivery_month`	`int`	1–12
`delivery_year`	`int`	e.g. `2026`
`expiry_date`	`date`	Options expiry
`T`	`float`	Time to expiry (years, ACT/365)
`forward_price`	`float`	EUR/MWh, rounded to 4 decimals
`source`	`str`	`"yahoo_finance"` or `"synthetic"`

3.4 `TTFForwardCurve`

Fetches and interpolates the TTF forward curve.

```
from datetime import date
from ttf_market_data import TTFForwardCurve

curve = TTFForwardCurve(
    reference_date=date(2026, 4, 20),
    risk_free_rate=0.03,
    timeout=10,
).load()
```

The constructor takes:

- `reference_date` : valuation date (defaults to `date.today()`)
- `risk_free_rate` : EUR risk-free rate used for cost-of-carry projection
- `timeout` : HTTP timeout in seconds for the Yahoo Finance request

`.load()` fetches the front-month TTF spot from Yahoo Finance (`ticker = "TTF=F"`); on failure (network error, throttling, schema change), it logs a warning and falls back to a representative synthetic spot of `35.0 EUR/MWh`. Each forward is then projected with a small seasonal premium:

```
seasonal(m) = 0.03 · sin(2π · (m − 1) / 12)
F(c) = spot · exp((r + seasonal(m)) · T)
```

`forward_price(T) -> float`

Linearly interpolated forward price for a tenor `T` (years). Flat extrapolation outside the loaded range.

```
curve.forward_price(0.25)   # -> 35.40 EUR/MWh   (illustrative)
curve.forward_price(1.00)   # -> 36.85 EUR/MWh
```

`to_dataframe() -> pandas.DataFrame`

Returns the full curve as a `DataFrame` with columns `contract_code, delivery_month, delivery_year, expiry_date, T, forward_price, source`.

```
curve.to_dataframe().head()
#   contract_code  delivery_month  delivery_year  expiry_date     T
forward_price       source
# 0       TTFK26               5           2026   2026-04-24  0.014
35.0145  yahoo_finance
# 1       TTFM26               6           2026   2026-05-22  0.090
35.1842  yahoo_finance
# ...
```

> ***Tip*** *: if Yahoo Finance is unavailable in your network (corporate proxy, sandboxed CI), wrap the load in a* `try/except` *— the module already logs a warning but still returns a usable synthetic curve.*

3.5 `VolSmile` and `VolatilitySurface`

```
@dataclass
class VolSmile:
    T: float                  # time to expiry (years)
    contract_code: str        # e.g. 'May26'
    F: float                  # forward at this tenor
    strikes: list[float]      # 9 strikes by default
    vols: list[float]         # Black-76 lognormal vols
    model: str = "black76"    # "black76" or "bachelier"
```

`VolatilitySurface` is a container of smiles indexed by maturity. It provides:

- `add_smile(smile)` — append a smile (and invalidate the bilinear cache)
- `vol(K, T) -> float` — bilinear interpolation in `(strike, maturity)` space
- `to_dataframe() -> DataFrame` — long format (one row per `(T, K)`)

```
v = surface.vol(K=32.0, T=0.50)        # interpolated lognormal vol
df = surface.to_dataframe()             # for plotting / export
```

> *The interpolator is built on first call to `.vol(...)` and re-cached automatically when a new smile is appended.*

3.6 `VolatilitySurfaceBuilder`

Builds a parametric TTF vol surface when no live option quotes are available.

```
from ttf_market_data import VolatilitySurfaceBuilder

builder = VolatilitySurfaceBuilder(
    forward_curve=curve,
    reference_date=date(2026, 4, 20),
    n_strikes=9,
)
surface = builder.build()
```

Inputs (all optional with sensible defaults):

Argument	Default	Meaning
`atm_vols`	`{1/12: 0.65, 2/12: 0.58, 3/12: 0.52, 6/12: 0.46, 9/12: 0.42, 1.0: 0.40, 2.0: 0.38}`	ATM lognormal vol per tenor
`rr25`	`-0.03` per tenor	25-delta risk-reversal (negative ⇒ put vol > call vol)
`bf25`	`+0.015` per tenor	25-delta butterfly (smile convexity)

For each tenor, the builder converts (ATM, RR25, BF25) into 25Δ call and 25Δ put vols, inverts those to strikes via `_delta_to_strike`, and interpolates the vols along the resulting strike grid in log-moneyness space:

```
σ_25c = ATM + 0.5 · RR25 + BF25
σ_25p = ATM − 0.5 · RR25 + BF25
```

If the forward at a given tenor falls below `2 EUR/MWh`, the smile is tagged `model="bachelier"` so downstream code knows to switch pricing models.

3.7 `SABRParams` and `MarketCalibration`

```
@dataclass
class SABRParams:
    alpha: float   # initial vol level
    beta: float    # CEV exponent (0.5 — typical for energy markets)
    rho: float     # vol-spot correlation
    nu: float      # vol-of-vol
```

`MarketCalibration` runs SABR (Hagan et al. 2002) on every Black-76 smile in the surface; Bachelier smiles are skipped (with a warning).

```
from ttf_market_data import MarketCalibration

calib = MarketCalibration(surface).calibrate_all()
calib.to_dataframe()
#       T   alpha  beta      rho      nu
# 0  0.083  0.471  0.50  -0.214  0.521
# 1  0.167  0.434  0.50  -0.198  0.488
# ...
```

Beta is fixed at `0.5`. The optimiser is `scipy.optimize.minimize` with L-BFGS-B and bounds `alpha ∈ [1e-4, 5]`, `rho ∈ [−0.999, 0.999]`, `nu ∈ [1e-4, 5]`.

3.8 Export utilities

Three helpers serialise to both CSV and JSON. `path` is the **prefix without extension**; the helpers append `.csv` / `.json`.

`export_forward_curve(curve, path)`

Writes `path.csv` and `path.json`. The JSON wraps the records under a top-level `{ "reference_date": "...", "curve": [...] }` envelope.

```
from ttf_market_data import export_forward_curve

export_forward_curve(curve, "ttf_output/ttf_forward_curve")
# -> ttf_output/ttf_forward_curve.csv
# -> ttf_output/ttf_forward_curve.json
```

`export_vol_surface(surface, path)`

Writes three files: `path.csv` (long format), `path_pivot.csv` (wide pivot, rows = `T`, columns = strike), and `path.json` (records under `{ "reference_date": "...", "surface": [...] }`).

`export_sabr_params(calibration, path)`

Writes `path.csv` and `path.json` with one row per calibrated tenor.

`export_all(output_dir=".", reference_date=None, risk_free_rate=0.03)`

Runs the full pipeline (forward curve → vol surface → SABR calibration) and writes everything to `output_dir`. Returns a `dict` with the three DataFrames for in-process use.

```
from ttf_market_data import export_all
from datetime import date

dfs = export_all(output_dir="ttf_output", reference_date=date(2026, 4, 20))
sorted(dfs.keys())
# ['forward_curve', 'sabr_params', 'vol_surface']
```

This is the recommended one-shot entry point for the rest of the project — the dashboard, the spread pricer and the structures module all consume the JSON files written here.

3.9 End-to-end example

```
from datetime import date
from ttf_market_data import (
    TTFExpiryCalendar, TTFForwardCurve,
    VolatilitySurfaceBuilder, MarketCalibration,
)

ref = date(2026, 4, 20)

# 1. Calendar — next 6 active TTF contracts
cal = TTFExpiryCalendar(ref)
for c in cal.active_contracts(n=6):
    print(f"{c.contract_code}  expiry={c.expiry_date}  T={c.T:.4f}y")

# 2. Forward curve (Yahoo Finance, with synthetic fallback)
curve = TTFForwardCurve(ref, risk_free_rate=0.03).load()
print(curve.to_dataframe()[["contract_code", "T", "forward_price", "source"]])

# 3. Vol surface (parametric, ATM term-structure + 25Δ smile)
surface = VolatilitySurfaceBuilder(curve, ref).build()
print(surface.vol(K=32.0, T=0.25))   # interpolated lognormal vol

# 4. SABR calibration (per tenor)
calib = MarketCalibration(surface).calibrate_all()
print(calib.to_dataframe())
```

---

4. `ttf_hh_spread.py`

A pricer for **exchange options** on the TTF − HH basis using **Margrabe's formula** (1978). The module covers:

- Unit conversion EUR/MWh ⇄ USD/MMBtu (`1 MWh = 3.412142 MMBtu`)
- Margrabe call/put on `(F_TTF, F_HH)` in USD/MMBtu
- All first-order Greeks plus `vega_ρ` (sensitivity to correlation)
- Implied correlation solver (Brent, monotone in ρ)
- Correlation- and vol-sensitivity tables
- A formatted summary printer

> ***Conventions for this section*** *- TTF forward:* `F_ttf_eur = 30 EUR/MWh` *- Henry Hub forward:* `F_hh = 3 USD/MMBtu` *- EUR/USD:* `fx_eurusd = 1.08` *- Vols:* `σ_TTF = 0.60` *(60%),* `σ_HH = 0.50` *(50%) - Correlation:* `ρ = 0.35` *- USD risk-free:* `r_usd = 0.045`

4.1 Background — exchange options and Margrabe

For two log-normal forwards `F_TTF`, `F_HH` (both in **USD/MMBtu**) with vols `σ_TTF`, `σ_HH` and instantaneous correlation `ρ`, the **spread vol** is:

```
σ_s = √(σ_TTF² + σ_HH² − 2·ρ·σ_TTF·σ_HH)
```

The price of the exchange call (Margrabe, 1978) is:

```
C = e^(-rT) · [F_TTF · N(d₁) − F_HH · N(d₂)]
P = e^(-rT) · [F_HH · N(−d₂) − F_TTF · N(−d₁)]

with  d₁ = [ln(F_TTF / F_HH) + ½·σ_s²·T] / (σ_s·√T)
      d₂ = d₁ − σ_s·√T
```

Intuition:

- **ρ → +1** (assets co-move) → `σ_s ↓` → option **cheaper**
- **ρ → −1** (assets diverge) → `σ_s ↑` → option **more expensive**
- The implied correlation backed out of a market quote is the market's view of TTF/HH co-movement.

Typical LNG-driven regime (2022–2026):

Quantity	Range
TTF forward	25 – 45 EUR/MWh (≈ 8 – 14 USD/MMBtu after FX)
Henry Hub forward	2 – 5 USD/MMBtu
Spread TTF − HH	5 – 12 USD/MMBtu (LNG netback drives convergence)
Implied correlation	≈ 0.20 – 0.55

4.2 Constants and unit conversions

`MWH_TO_MMBTU = 3.412142`

Exact energy equivalence factor: `1 MWh = 3.412142 MMBtu`.

`ttf_eur_to_usd(F_ttf_eur, fx_eurusd) -> float`

Converts a TTF forward from EUR/MWh to USD/MMBtu: `F_usd = F_eur · fx_eurusd / 3.412142`.

```
from ttf_hh_spread import ttf_eur_to_usd

ttf_eur_to_usd(F_ttf_eur=30.0, fx_eurusd=1.08)
# -> 9.4955   USD/MMBtu
```

`ttf_usd_to_eur(F_ttf_usd, fx_eurusd) -> float`

Inverse conversion.

```
from ttf_hh_spread import ttf_usd_to_eur

ttf_usd_to_eur(F_ttf_usd=10.0, fx_eurusd=1.08)
# -> 31.5939   EUR/MWh
```

`spread_usd_to_eur(spread_usd, fx_eurusd) -> float`

Converts any USD/MMBtu spread or option premium back into EUR/MWh.

```
from ttf_hh_spread import spread_usd_to_eur

spread_usd_to_eur(spread_usd=1.0, fx_eurusd=1.08)
# -> 3.1594    EUR/MWh
```

4.3 Margrabe pricing — core

The functions in this section work in **USD/MMBtu** for both forwards. Use them when the TTF forward has already been converted (or for unit testing).

`margrabe_price(F_ttf, F_hh, T, r, sigma_ttf, sigma_hh, rho, option_type='call') -> float`

```
from ttf_hh_spread import margrabe_price

# ATM-like spread: F_TTF ≈ F_HH (USD/MMBtu)
margrabe_price(F_ttf=9.50, F_hh=9.00, T=0.25, r=0.045,
               sigma_ttf=0.60, sigma_hh=0.50, rho=0.35,
               option_type="call")
# -> 1.4129   USD/MMBtu

margrabe_price(9.50, 9.00, 0.25, 0.045, 0.60, 0.50, 0.35, "put")
# -> 0.9185   USD/MMBtu
```

> ***Edge cases**: when `T ≤ 0` or `σ_s < 1e-12`, the function returns the discounted intrinsic `e^(-rT)·max(F_TTF − F_HH, 0)` (call) or its symmetric counterpart (put).*

Raises `ValueError` if `option_type` ∉ `{'call', 'put'}`.

`margrabe_greeks(F_ttf, F_hh, T, r, sigma_ttf, sigma_hh, rho, option_type='call') -> SpreadGreeks`

All first-order sensitivities returned in a single `SpreadGreeks` dataclass:

Field	Definition	Unit
`delta_ttf`	`∂Price/∂F_TTF`	dimensionless (∈ [0,1] for call)
`delta_hh`	`∂Price/∂F_HH`	dimensionless (∈ [−1,0] for call)
`gamma_ttf`	`∂²Price/∂F_TTF²`	per (USD/MMBtu)
`vega_ttf`	`∂Price/∂σ_TTF`	USD/MMBtu per 1.00 of vol
`vega_hh`	`∂Price/∂σ_HH`	USD/MMBtu per 1.00 of vol
`vega_rho`	`∂Price/∂ρ`	USD/MMBtu per unit correlation
`theta`	1-day finite-difference decay	USD/MMBtu per calendar day

```
from ttf_hh_spread import margrabe_greeks

g = margrabe_greeks(F_ttf=9.50, F_hh=9.00, T=0.25, r=0.045,
                    sigma_ttf=0.60, sigma_hh=0.50, rho=0.35,
                    option_type="call")
g.delta_ttf   # +0.6219
g.delta_hh    # -0.4995
g.gamma_ttf   # +0.1244
g.vega_ttf    # +1.1928   (per 1.00 of vol)
g.vega_hh     # +0.8139
g.vega_rho    # -0.8420   (negative: ↑ρ ⇒ ↓price)
g.theta       # -0.0060   (per calendar day)
```

> ***Vega convention**: per unit of decimal vol (1.00 = 100%). Divide by 100 for the "per 1 vol point" market convention.*
>
> ***`vega_rho` sign**: a higher `ρ` reduces `σ_s`, hence reduces the option premium. `vega_rho` is therefore **negative** for both call and put.*
>
> ***Theta**: computed by 1-day finite difference (`T → T − 1/365`), so already expressed per **calendar day**. Negative for a long position with positive time value.*

4.4 Full pricer (TTF in EUR/MWh)

End-to-end entry point. It accepts TTF in **EUR/MWh**, performs the FX and energy-unit conversion internally, and returns a `SpreadResult` carrying the prices in **both** USD/MMBtu and EUR/MWh, plus all the Greeks.

`spread_price(F_ttf_eur, F_hh, fx_eurusd, T, r_usd, sigma_ttf, sigma_hh, rho, option_type='call', reference=None) -> SpreadResult`

```
from ttf_hh_spread import spread_price

res = spread_price(
    F_ttf_eur=30.0, F_hh=3.0, fx_eurusd=1.08,
    T=0.25, r_usd=0.045,
    sigma_ttf=0.60, sigma_hh=0.50, rho=0.35,
    option_type="call",
)

res.F_ttf_usd      # 9.4955
res.sigma_spread   # 0.6325   (= 63.25%)
res.price          # 6.4229   USD/MMBtu
res.price_eur      # 20.2924  EUR/MWh
res.greeks.delta_ttf
```

The `T` argument may also be a **TTF contract code** (`"TTFM26"` , `"Jun26"` , `"Mar2026"`); it is then resolved via `t_from_contract` from `black76_ttf.py`.

```
from datetime import date
from ttf_hh_spread import spread_price

res = spread_price(
    F_ttf_eur=30.0, F_hh=3.0, fx_eurusd=1.08,
    T="TTFM26", r_usd=0.045,
    sigma_ttf=0.60, sigma_hh=0.50, rho=0.35,
    option_type="call",
    reference=date(2026, 4, 1),
)
# T          -> 0.1425   (52 days / 365)
# price      -> 6.4540   USD/MMBtu
# price_eur  -> 20.3907  EUR/MWh
```

The `SpreadResult` dataclass also carries the normalised inputs (`F_ttf_eur` , `F_ttf_usd` , `F_hh` , `fx_eurusd` , `sigma_ttf` , `sigma_hh` , `rho` , `sigma_spread` , `T` , `r_usd` , `option_type`) for later inspection.

4.5 Implied correlation

Brent solver, `xtol = 1e-8`, max 300 iterations. The premium is **monotone-decreasing in ρ**, so the bracket always converges if the market price lies inside the achievable corridor.

`implied_correlation(market_price, F_ttf, F_hh, T, r, sigma_ttf, sigma_hh, option_type='call', rho_lo=-0.9999, rho_hi=0.9999) -> float`

```
from ttf_hh_spread import margrabe_price, implied_correlation

# Price an ATM spread at ρ = 0.5, then back out ρ from the price
mp = margrabe_price(9.50, 9.50, 0.25, 0.045, 0.60, 0.50, 0.50, "call")
# -> 1.0399

implied_correlation(mp,
                    F_ttf=9.50, F_hh=9.50,
                    T=0.25, r=0.045,
                    sigma_ttf=0.60, sigma_hh=0.50,
                    option_type="call")
# -> 0.5000
```

Raises `ValueError` if `market_price` lies outside the achievable corridor `[price@ρ=rho_hi, price@ρ=rho_lo]`. The error message reports the bounds — typically a quote below this corridor signals a price below intrinsic.

4.6 Sensitivity helpers

`rho_sensitivity(F_ttf, F_hh, T, r, sigma_ttf, sigma_hh, option_type='call', rhos=None) -> list[(rho, price)]`

Default ρ-grid: `[-0.9, -0.7, -0.5, -0.3, -0.1, 0.0, 0.1, 0.3, 0.5, 0.7, 0.9]`.

```
from ttf_hh_spread import rho_sensitivity

rho_sensitivity(F_ttf=9.50, F_hh=9.00, T=0.25, r=0.045,
                sigma_ttf=0.60, sigma_hh=0.50,
                option_type="call",
                rhos=[-0.5, 0.0, 0.35, 0.7])
# -> [(-0.5, 1.9821), (0.0, 1.6765), (0.35, 1.4129), (0.7, 1.0651)]
```

The price is **monotone-decreasing in ρ**, as expected for an exchange option.

`vol_sensitivity(F_ttf, F_hh, T, r, sigma_ttf, sigma_hh, rho, option_type='call') -> dict`

One-way ±5 vol-point bumps for each of `σ_TTF`, `σ_HH`, plus a ±0.10 bump on ρ — useful for a quick risk dashboard.

```
from ttf_hh_spread import vol_sensitivity

vol_sensitivity(F_ttf=9.50, F_hh=9.00, T=0.25, r=0.045,
                sigma_ttf=0.60, sigma_hh=0.50, rho=0.35,
                option_type="call")
# {'base':        1.4129,
#  'σ_ttf +5%':   1.4744,
#  'σ_ttf −5%':   1.3553,
#  'σ_hh  +5%':   1.4563,
#  'σ_hh  −5%':   1.3751,
#  'ρ     +0.10': 1.3253,
#  'ρ     −0.10': 1.4942}
```

4.7 Display helper

`print_summary(res: SpreadResult) -> None`

Pretty-prints a full pricing report (underlyings, vols, premiums in USD/MMBtu **and** EUR/MWh, all Greeks, and Vega in the "per 1% vol move" market convention).

```
from ttf_hh_spread import spread_price, print_summary

res = spread_price(30.0, 3.0, 1.08,
                   T=180/365, r_usd=0.045,
                   sigma_ttf=0.60, sigma_hh=0.50, rho=0.35,
                   option_type="call")
print_summary(res)
# ═══════════════════════════════════════════════════════════════
#   TTF / Henry Hub Spread Option — CALL
# ═══════════════════════════════════════════════════════════════
#   ...
#   Option premium   :   6.3563 USD/MMBtu
#   Option premium   :  20.0821 EUR/MWh
#   ...
```

4.8 Put-call parity

For Margrabe exchange options, parity reads:

```
C − P = e^(-rT) · (F_TTF − F_HH)
```

Verification on the demo case (`F_TTF = 9.4955` , `F_HH = 3.0` , `T = 180/365` , `r = 0.045` , `σ_TTF = 0.60` , `σ_HH = 0.50` , `ρ = 0.35`):

Side	Value (USD/MMBtu)
`C − P`	6.352943
`e^(-rT)·(F_TTF − F_HH)`	6.352943

PCP holds to better than `1e-8 USD/MMBtu`.

---

5. `dashboard_jupyter.ipynb`

Interactive **Jupyter dashboard** built on top of the four pricing modules. Each section is a standalone widget panel: tweak the sliders/dropdowns and the prices, Greeks and charts re-render automatically.

5.0 Launching the notebook

```
# from the project root
pip install -r requirements.txt   # numpy, scipy, matplotlib, pandas,
                                  # ipywidgets, jupyterlab
jupyter lab dashboard_jupyter.ipynb
```

Then run the cells from top to bottom (`Shift + Enter`). Dependencies pulled in by the notebook:

Module	Used by
`black76_ttf`	Section 1 (Pricer), Section 5 (expiries)
`structures_ttf`	Section 2 (multi-leg structures)
`ttf_hh_spread`	Section 4 (Margrabe spread)
`ipywidgets`, `matplotlib`, `pandas`, `numpy`	UI + plots + tables

> ***Tip**: each section reads only its own widgets — you can edit / collapse any one of them without breaking the others.*

5.1 Section 1 — Pricer (Black-76 / Bachelier)

Single-leg European call/put pricer with full Greeks.

Inputs

Widget	Description	Default
`Model`	`Black-76` (lognormal) or `Bachelier` (normal)	Black-76
`Forward (EUR/MWh)`	Underlying TTF forward `F`	35.00
`Strike (EUR/MWh)`	Exercise price `K`	35.00
`Maturity (years)`	`T` , ACT/365	0.25
`Rate (decimal)`	Risk-free rate `r` (annualised)	0.03
`Vol (lognormal)`	Black-76 σ (visible only when `Model = Black-76`)	0.50
`Vol (EUR/MWh)`	Bachelier σ_n (visible only when `Model = Bachelier`)	10.00

Outputs

- Call and put premiums in EUR/MWh.
- Full Greek block for both call and put: `delta` , `gamma` , `vega` , `theta` (per calendar day), `rho` (per percentage point), `vanna` , `volga` .

Behaviour

- The vol slider auto-toggles between lognormal (Black-76) and normal (Bachelier) when the model dropdown changes — only the relevant one is shown.
- Recomputation is triggered on slider release (`continuous_update=False`), so the panel stays responsive on slow links.
- Internally calls `b76_call` , `b76_put` , `b76_greeks` (resp. `bach_*`) from `black76_ttf.py` .

5.2 Section 2 — Structures (10 multi-leg payoffs)

Pricer + P&L chart for the standard option structures wrapped in `structures_ttf.py` . Black-76 only, fixed `r = 3 %` .

Available structures

#	Structure	Strike inputs
1	Straddle	`K`
2	Strangle	`K_put`, `K_call`
3	Bull Call Spread	`K_lo`, `K_hi`
4	Bear Put Spread	`K_lo`, `K_hi`
5	Butterfly	`K_lo`, `K_mid`, `K_hi`
6	Condor	`K1`, `K2`, `K3`, `K4`
7	Collar	`K_put`, `K_call`
8	Risk Reversal	`K_put`, `K_call`
9	Calendar Spread	`K` (`T_far = T + 3 months`)
10	Ratio Spread 1×2	`K_lo`, `K_hi`

Inputs

- `Structure` dropdown — picking a new structure resets the strike sliders to sensible defaults around the forward (e.g. `F − 4, F, F + 4` for a butterfly).
- `Forward (EUR/MWh)`, `Maturity (years)`, `Vol (lognormal)`.
- 1 to 4 strike sliders `K1 … K4` whose **labels and visibility adapt** to the selected structure.

Outputs

- **Net premium** in EUR/MWh, labelled `debit` if positive, `credit` if negative.
- **Net Greeks**: `delta`, `gamma`, `vega`, `theta` (per calendar day).
- **Breakevens** (forward levels at expiry where P&L = 0).
- **Max profit** / **max loss** (`+inf` / `−inf` for unbounded structures).
- **P&L chart at expiry** with:
- solid blue line = total payoff − net premium,
- dashed grey line = current forward,
- dotted red lines = breakevens,
- light green vertical lines = strikes,
- green/red shading = profit/loss zones.

> ***Tip**: Calendar Spread is the only structure that uses two maturities — the slider drives `T_near`, while `T_far = T_near + 0.25 yr` is fixed inside the dispatcher.*

5.3 Section 3 — Vol Surface (parametric, 3D)

Parametric implied volatility surface, displayed as an interactive 3D `matplotlib` plot.

Model

ATM term structure plus a smile in log-moneyness:

```
atm(T)     = sigma_inf + delta_sigma · exp(−kappa · T)
m(K, T)    = ln(K / F) / sqrt(T)
sigma(K,T) = atm(T) + skew · m + wings · m²        (clipped to [0.01, 5.00])
```

The forward is held constant across the whole grid (a true forward curve could be plugged in via `ttf_market_data` later).

Inputs

Widget	Meaning	Range
`Forward (EUR/MWh)`	`F` (constant across the grid)	5 – 100
`sigma_inf (long ATM)`	long-end ATM vol	0.10 – 1.00
`delta_sigma (short ATM)`	additional short-end ATM bump	0.00 – 1.00
`kappa (decay)`	exponential decay rate of the bump	0.1 – 8.0
`skew`	linear smile slope vs log-moneyness	−0.40 – +0.40
`wings (convexity)`	quadratic smile coefficient	0.00 – 0.50
`range strikes (± %)`	strike grid half-width vs `F`	10 % – 90 %
`T max (years)`	longest maturity on the surface	0.25 – 5.0

Outputs

- A 3D surface plot (axes: `K`, `T`, `σ`) with the `viridis` colormap and a colourbar.
- Two info lines below the chart: `ATM(T=0)`, `ATM(T=∞)`, the achieved vol range and the grid size (default 35 strikes × 25 maturities).

> ***Tip**: a negative `skew` (default `-0.08`) reproduces the classic TTF shape — OTM puts trade richer than OTM calls. Increase `wings` to fatten both tails.*

5.4 Section 4 — TTF / Henry Hub Spread (Margrabe)

UI wrapper around `ttf_hh_spread.spread_price` (see Part 4). Prices and plots both the **payoff at expiry** and the **current option value** along the TTF axis, with a secondary x-axis in EUR/MWh.

Inputs

Widget	Description	Default
`Option`	`call` or `put`	call
`Forward TTF (EUR/MWh)`	TTF forward in EUR/MWh	35.00
`Forward HH (USD/MMBtu)`	Henry Hub forward in USD/MMBtu	3.50
`Vol TTF (lognormal)`	σ_TTF	0.50
`Vol HH (lognormal)`	σ_HH	0.45
`Correlation rho`	TTF/HH correlation ρ	0.50
`Maturity (years)`	`T`, ACT/365	0.50
`FX EUR/USD`	spot FX used to convert TTF to USD/MMBtu	1.080
`Rate USD`	USD risk-free rate	0.04
`Market price (USD/MMBtu)`	optional input for implied correlation	0.00

Outputs

- TTF forward shown in **both** EUR/MWh and USD/MMBtu, plus the spread vol `σ_spread = √(σ_TTF² + σ_HH² − 2·ρ·σ_TTF·σ_HH)`.
- Premium in **both** USD/MMBtu and EUR/MWh.
- All Margrabe Greeks: `delta_ttf`, `delta_hh`, `gamma_ttf`, `vega_ttf`, `vega_hh`, `vega_rho`, `theta` per calendar day.
- **Implied correlation**: if `Market price > 0`, the panel solves `implied_correlation` and prints the result. If the quote is outside the achievable corridor, an explanatory error is shown instead.
- A **payoff vs option-value chart**:
- blue solid line = payoff at expiry vs `F_TTF` (USD/MMBtu),
- orange dashed line = current Margrabe value at the chosen `T`,
- vertical reference lines at `F_TTF` (grey) and `F_HH` (red),
- top x-axis = `F_TTF` translated back into EUR/MWh.

5.5 Section 5 — Expiry Dates (ICE Endex Dutch TTF)

Calendar viewer for the official ICE Endex TTF option expiries.

Rule: `expiry = (1st of delivery month) − 5 calendar days`; if not a business day, roll back to the previous business day (NL + UK holidays); if equal to the futures LTD, roll back one extra business day.

Inputs

Widget	Description	Default
`Number of contracts`	how many forthcoming expiries to display	12
`Reference`	reference date for the lookup	today

Outputs

A `pandas` DataFrame with one row per contract:

Column	Meaning
`Contract`	ICE code (`TTFK26` , `TTFM26` , …)
`Delivery`	delivery month (`May-26` , `Jun-26` , …)
`Option expiry`	ISO date of the option expiry
`Days to expiry`	`(expiry − reference) + 1` calendar days
`T (years)`	`T = Days / 365`
`Futures LTD`	last trading day of the underlying future

A **horizontal bar chart** displays `T (years)` for each contract, annotated with the calendar-day count.

> ***Backed by***: *`ttf_next_expiries`, `ttf_time_to_expiry`, `_ttf_futures_ltd` from `black76_ttf.py`.*

---

6. Financial glossary

Every market, model, Greek, structure and numerical convention used in the project, grouped by topic. Cross-references in *italics* point to other glossary entries.

6.1 Markets and underlyings

- **TTF** (*Title Transfer Facility*) — Dutch virtual natural gas hub operated by Gasunie Transport Services. The European benchmark for natural gas.
- **Henry Hub (HH)** — Physical natural gas pipeline interconnection in Erath, Louisiana. The US benchmark, settlement reference for NYMEX natural gas futures.
- **ICE Endex** — Exchange listing the *TTF* futures and options. Sets the official expiry calendar (Dutch + UK holiday rules).

- **Forward** — Agreed price today for delivery on a future date. Notation `F`. Implicitly already includes carry and interest, hence Black-76 prices options on `F` rather than on a spot price.
- **Future / Futures contract** — Exchange-listed, marked-to-market version of a forward. One TTF futures contract = physical delivery over a given delivery month.
- **Delivery month** — The calendar month over which a TTF futures contract delivers gas (e.g. `Jun26` → June 2026).
- **LTD** (*Last Trading Day*) — The last business day on which the future itself can be traded. The TTF option expires **before** the futures LTD.
- **LNG** (*Liquefied Natural Gas*) — Cross-basin transportable form of natural gas. LNG flows between TTF and HH drive the spread vol via the *netback* relationship.
- **Netback** — Arbitrage-style equality between two regional prices net of liquefaction, shipping and regasification costs. Drives convergence of TTF and HH and therefore the *implied correlation* on the spread.

6.2 Option mechanics

- **Option** — Right (without the obligation) to transact an underlying at a fixed price.
- **Call** — Option to **buy** the underlying at strike `K`. Payoff at expiry: `max(F_T − K, 0)`.
- **Put** — Option to **sell** the underlying at strike `K`. Payoff at expiry: `max(K − F_T, 0)`.
- **European style** — Exercise allowed **only at expiry**. The whole library assumes European options.
- **American style** — Exercise allowed at any time up to expiry (not used here).
- **Strike** (`K`) — Pre-agreed exercise price.
- **Maturity / Expiry** (`T`) — Date / remaining time at which the option can be exercised. Expressed in years (*ACT/365*).
- **Premium** — Price paid for the option (in EUR/MWh or USD/MMBtu).
- **Debit / Credit** — A multi-leg structure with a *positive* net premium (cost) is a *debit*; with a *negative* net premium (proceeds) it is a *credit*.
- **Intrinsic value** — Immediate exercise value: `max(F − K, 0)` (call) or `max(K − F, 0)` (put). What an option would be worth if expiry were now.
- **Time value** — `Premium − Intrinsic`. Positive while $T > 0$.
- **ATM** (*At-The-Money*) — `K = F`.
- **ITM** (*In-The-Money*) — Call: `K < F`. Put: `K > F`. Positive intrinsic.
- **OTM** (*Out-of-The-Money*) — Call: `K > F`. Put: `K < F`. Zero intrinsic.
- **Moneyness** — Generic ratio describing the relationship between `K` and `F`. *Log-moneyness* `m = ln(K / F) / √T` is used by the vol smile.
- **Payoff** — Cashflow at expiry as a function of the terminal forward `F_T`.

- **P&L at expiry** — Payoff *net of the premium paid*, used in *Section 2* of the dashboard.
- **Breakeven** — Forward level at expiry at which net P&L = 0.
- **Put-call parity (PCP)** — Model-free identity: `C − P = e^(−rT) · (F − K)`. Holds for both Black-76 and Bachelier; the Margrabe analogue is `C − P = e^(−rT) · (F_TTF − F_HH)`.

6.3 Pricing models

- **Black-Scholes (1973)** — Foundational lognormal model for stock options with continuous dividends.
- **Black-76 (1976)** — Adaptation of Black-Scholes for options on **forwards** (no spot, no carry): `C = e^(−rT)·[F·N(d₁) − K·N(d₂)]` with `d₁ = [ln(F/K) + ½σ²T] / (σ√T)`, `d₂ = d₁ − σ√T`. Default model in this project for `F ≫ 0`.
- **Bachelier (1900)** — Normal (arithmetic) Brownian-motion model: `dF = σₙ · dW`. Allows **negative forwards**, hence the right model for TTF in crisis or for spreads.
- **Margrabe (1978)** — Closed-form pricer for an *exchange option* on two log-normal forwards. See *Spread vol*.
- **Risk-neutral measure** — Probability measure under which discounted forwards are martingales. All pricers in the library evaluate expectations under that measure.
- **Discount factor (DF)** — `e^(−rT)`. Multiplies the forward expectation to convert it into a present value.
- **Risk-free rate** (`r`) — Annualised continuously compounded rate. Decimal (`0.02 = 2 %`).
- **No-arbitrage bounds** — A call price must lie in `[max(0, e^(−rT)(F−K)), e^(−rT)·F]`. The implied-vol solvers raise `ValueError` if a quote breaches them.
- **N(·) / Φ(·)** — Standard-normal CDF.
- **φ(·) / n(·)** — Standard-normal PDF.

6.4 The Greeks

Sensitivities of the option price `V` with respect to its inputs.

- **Delta (Δ)** — `∂V / ∂F`. Hedge ratio in forward units. Discounted by `e^(−rT)` here. `Δ_call − Δ_put = e^(−rT)`.
- **Gamma (Γ)** — `∂²V / ∂F²`. Convexity in the forward; identical for call and put. Peaks ATM and decays toward expiry.
- **Vega (ν)** — `∂V / ∂σ`. Sensitivity to volatility. Reported here per unit (1.00 = 100 %) of vol; divide by 100 for the market "per 1 vol point" convention.
- **Theta (Θ)** — `∂V / ∂t`. Time decay. Reported **per calendar day**. Negative for a long option.

- **Rho (ρ)** — `∂V / ∂r`. Sensitivity to the risk-free rate. Reported per *percentage point* of rate change.
- **Vanna** — `∂²V / (∂F ∂σ) = ∂Δ / ∂σ`. Cross-derivative used in skew-aware risk management. Zero ATM under Bachelier.
- **Volga** (also *Vomma*) — `∂²V / ∂σ²`. Vol convexity, drives the smile-roll risk.
- **Charm** / **Color** / **Speed** — Higher-order Greeks not used here but occasionally referenced in the literature.
- **Net Greeks** — Greeks aggregated across the legs of a multi-leg structure (Section 2 of the dashboard).

6.5 Volatility

- **Volatility (σ)** — Standard deviation of returns of the underlying.
- **Lognormal vol** — σ used in *Black-76*; dimensionless, decimal (`0.50 = 50 %`).
- **Normal vol** (σ_n) — σ used in *Bachelier*; absolute units, EUR/MWh. Rule of thumb: `σₙ ≈ F · σ` for matching ATM prices.
- **Implied volatility (IV)** — Vol level that, plugged into the model, reproduces a quoted market price. Solved by *Brent's method* in the library (`b76_implied_vol`, `bach_implied_vol`).
- **ATM volatility** — IV at `K = F`.
- **Smile** — Convex shape of `IV(K)` as a function of strike (or log-moneyness).
- **Skew** — Slope of the smile. The TTF smile is typically negatively skewed (OTM puts trade richer than OTM calls).
- **Wings** — Quadratic / convex contribution to the smile, fattening the tails on both sides.
- **Term structure** — Variation of ATM IV with maturity `T`.
- **Vol surface** — Two-dimensional `IV(K, T)`. Section 3 of the dashboard builds a parametric one: `σ(K,T) = sigma_inf + Δσ·exp(−κT) + skew·m + wings·m²`.
- **25-delta / 50-delta / 75-delta strike** — Strike whose call delta equals the given target. Standard quote convention; obtained via `b76_delta_to_strike` / `bach_delta_to_strike`.
- **Risk reversal** — Difference of IVs between the 25Δ call and the 25Δ put; measures *skew*. Also the name of a multi-leg structure (see 6.7).

6.6 Spread and correlation

- **Spread** — Difference of two related prices. Here `F_TTF − F_HH` after unit conversion to USD/MMBtu.
- **Spread option** — Option whose payoff depends on a spread, e.g. `max(F_TTF − F_HH, 0)`.

- **Exchange option** — Option to swap one asset for another at expiry, i.e. `max(F₁ − F₂, 0)`. Margrabe (1978) gives a closed form.
- **Margrabe formula** — Closed-form price of an exchange option: `C = e^(−rT)·[F₁·N(d₁) − F₂·N(d₂)]` with `d₁ = [ln(F₁/F₂) + ½σₛ²T] / (σₛ√T)`, `d₂ = d₁ − σₛ√T`.
- **Spread vol (σ_s)** — Effective lognormal vol of the spread: `σₛ = √(σ₁² + σ₂² − 2·ρ·σ₁·σ₂)`. Used inside Margrabe.
- **Correlation (ρ)** — Instantaneous correlation between the two log-forwards. ρ → +1 → assets co-move → spread vol shrinks → option cheaper.
- **Implied correlation** — ρ backed out of a market spread-option price via `implied_correlation` (Brent on a monotone function of ρ).
- **MWh / MMBtu** — Energy units. `1 MWh = 3.412142 MMBtu`. TTF quotes in EUR/MWh, HH in USD/MMBtu, hence the conversion in `ttf_eur_to_usd` / `ttf_usd_to_eur`.
- **FX EUR/USD** — Exchange rate (e.g. `1.08`). Combined with the energy conversion to compare TTF and HH on the same axis.

6.7 Multi-leg structures

Eleven structures bundled by `structures_ttf.py` and exposed in Section 2 of the dashboard.

- **Straddle** — Long call + long put at the **same** strike `K`. Pure vol bet, max profit unbounded, max loss = total premium.
- **Strangle** — Long OTM put `K_put` + long OTM call `K_call`. Cheaper than a straddle, requires a wider move to break even.
- **Bull Call Spread** — Long call `K_lo` + short call `K_hi`. Bounded bullish payoff for a *debit*.
- **Bear Put Spread** — Long put `K_hi` + short put `K_lo`. Bounded bearish payoff for a *debit*.
- **Butterfly** — Long `K_lo` + short 2 × `K_mid` + long `K_hi` (all calls or all puts). Pin-risk play, max profit at `K_mid`.
- **Condor** — Four-strike generalisation of the butterfly: long `K1` + short `K2` + short `K3` + long `K4`. Wide pin zone.
- **Collar** — Long put `K_put` + short call `K_call`. Caps both upside and downside, near-zero premium for a hedger of the underlying.
- **Risk Reversal** — Short put `K_put` + long call `K_call`. Synthetic long forward; price reflects the *skew*.
- **Calendar Spread** — Short near-dated option + long far-dated option at the same strike. Sells short-dated theta against long-dated vega.
- **Ratio Spread (1×2)** — Long 1 × `K_lo` + short 2 × `K_hi`. Cheap or even *credit* directional bet with naked-leg risk past `K_hi`.

- **Net premium** — Sum of premiums across the legs (sign-aware).
- **Breakevens / Max profit / Max loss** — Reported by the dashboard for each structure (`+inf` / `-inf` for unbounded structures).

6.8 Conventions and numerics

- **ACT/365** — Day-count convention used throughout: `T = days / 365`, including the reference day.
- **Business day** — Mon–Fri excluding NL + UK holidays. Implemented in `ttf_is_business_day` (1 January, Good Friday, Easter Monday, 1 May, 25 and 26 December).
- **ICE month codes** — `F G H J K M N Q U V X Z` for January–December. Combined with a 2-digit year: `TTFM26` = June 2026.
- **Reference date** — Date from which `T` is measured. Defaults to `date.today()` in the helpers.
- **Brent's method** — Bracketing root-finder used for *implied volatility* and *implied correlation* (`xtol = 1e-8`, max 300 iterations).
- **Finite-difference Theta** — Theta is computed by `V(T − 1/365) − V(T)` rather than analytically, to stay consistent across the four pricers.
- **Sign convention** — Greeks reported with their natural sign: long call Δ > 0, long option Θ < 0, etc.
- **Numerical accuracy** — All round-trip identities (PCP, IV → price → IV, spread PCP) are verified to better than `1e-8` in the built-in tests.

Appendix A — Reference values (3 months, ATM)

With `F = 30 EUR/MWh, K = 30, T = 0.25, r = 0.02`:

Model	sigma	Call	Put	Δ_call	Γ	ν
B76	0.50	2.9670	2.9670	+0.547	0.0525	5.907
Bach	15 EUR/MWh	2.9770	2.9770	+0.498	0.0530	0.199

Theoretical `C − P` = `e^(-0.02·0.25)·(30 − 30) = 0`. Verified on both models (difference < 1e-12 EUR/MWh).

Appendix B — Reference values for the TTF/HH spread (Margrabe)

With `F_ttf_eur = 30 EUR/MWh`, `F_hh = 3 USD/MMBtu`, `fx = 1.08`, `σ_TTF = 0.60`, `σ_HH = 0.50`, `ρ = 0.35`, `r_usd = 0.045`, `T = 180 / 365`:

Quantity	Value	Unit
`F_ttf_usd`	9.4955	USD/MMBtu
Spread `F_TTF − F_HH`	+6.4955	USD/MMBtu
`σ_spread`	63.25%	—
Call premium	6.3563	USD/MMBtu
Call premium	20.0821	EUR/MWh
Put premium	0.0034	USD/MMBtu
`Δ_TTF` (call)	+0.9757	—
`Δ_HH` (call)	−0.9694	—

PCP check: `C − P = 6.352943 = e^(-rT)·(F_TTF − F_HH)`, verified to `< 1e-8 USD/MMBtu`.